Strukturat e kontrollit në C

Programim në gjuhën C

Alion Emini

kurseinformatike.com

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

Përmbajtja

1. Hyrje
2. Cikli `while`
3. Cikli `do ... while`
4. Cikli `for`
5. `break`, `continue`, `goto`
6. Deklarata null
7. Mbyllje

Konteksti: nga strukturat e vendimmarrjes

Deri tani pamë:

- `if`, `if ... else`, `else if` për vendime;
- `switch` për shumë raste të ndara;
- vendimmarrje = zgjedhim *cilin bllok* kodi të ekzekutojmë.

Konteksti: nga strukturat e vendimmarrjes

Ideja e kapitullit të ri

Tani shtojmë **përsëritjen**: të ekzekutojmë të njëjtin bllok kodi **shumë herë**, derisa një kusht të ndryshojë.

Çfarë është një cikël?

Përkufizim

Një **cikël** është një deklaratë që ekzekuton në mënyrë të përsëritur një deklaratë tjetër (trupin e ciklit).

Në C, çdo cikël ka një **kusht** që testohet:

- nëse kushti është **TRUE** (jo-zero) $\Rightarrow$ vazhdon iteracioni;
- kur kushti bëhet **FALSE** (zero) $\Rightarrow$ cikli përfundon;
- një ekzekutim i trupit quhet **iteracion**.

Tre ciklet kryesore në C

- `while`: kushti testohet **në fillim** (0 herë ose më shumë).
- `do ... while`: kushti testohet **në fund** (të paktën 1 herë).
- `for`: formë kompakte për numërim (inicializim + kusht + përditësim).

Tre ciklet kryesore në C

Në këtë kapitull

Përveç cikleve, do të shohim edhe **kontrollin e rrjedhës**: `break`, `continue`, `goto` (me kujdes) dhe **deklaratën null** `;`.

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
4. Cikli `for`
5. `break`, `continue`, `goto`

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

Sintaksa e `while`

Forma bazë:

```
while (kushti) deklarate;
```

Kur trupi ka disa deklarata:

```
while (kushti) {
    deklarate;
    deklarate;
}
```

Si lexohet

"Për sa kohë që `kushti` është i vërtetë, ekzekuto trupin e ciklit."

Anatomia tipike e një `while`

Një `while` zakonisht ka këto pjesë:

- **Inicializimi** (p.sh. `i = 0;`)
- **Kushti** (p.sh. `i <= 10`)
- **Trupi** (p.sh. printimi, llogaritja, etj.)
- **Përditësimi** (p.sh. `i++;`) që e çon drejt përfundimit

Anatomia tipike e një `while`

Rregull praktik

Nëse nuk përditëson variablat që ndikojnë kushtin, rrezikon **cikël të pafund**.

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

Shembull: printimi i numrave 0 deri 10

```c
#include <stdio.h>

int main(void) {
    int i = 0;

    while (i <= 10) {
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```

Shembull: 0 deri 10 (si funksionon)

1. `i` inicializohet me `0`.
2. Para çdo iteracioni testohet `(i <= 10)`.
3. Trupi printon `i`.
4. `i++` e afron ciklin drejt përfundimit.

Shembull: 0 deri 10 (si funksionon)

E rëndësishme

Meqë kushti testohet në fillim, **trupi mund të mos ekzekutohet fare** nëse kushti është FALSE që në start.

Shembull: fuqia më e vogël e $2 \geq n$

```c
#include <stdio.h>

int main(void) {
    int n;
    int i = 1;

    printf("Jep n: ");
    scanf("%d", &n);

    while (i < n) {
        i = i * 2;
    }

    printf("Fuqia me e vogel e 2 >= %d eshte %d\n", n, i);
    return 0;
}
```

Shembull: fuqia e 2 (gjurmim i shkurtër)

Për `n = 10`, `i` kalon:

$$1 \to 2 \to 4 \to 8 \to 16$$

- kushti është `i < n`;
- sapo `i` bëhet 16, kushti bëhet FALSE dhe cikli ndalet;
- në fund `i` është vlera minimale e formës 2^k me $2^k \geq n$.

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

Cikël i pafund: `while (1)`

Në C, çdo vlerë jo-zero është TRUE, prandaj:

```
while (1) {
    /* trupi i ciklit */
}
```

Kujdes

Një cikël i pafund duhet të ketë **dalje të qartë** (p.sh. `break`, `return`, etj.).

Shembull: dalje nga cikli i pafund me break

```c
#include <stdio.h>

int main(void) {
    while (1) {
        int input;

        printf("Shkruani nje numer (0 per te dale): ");
        scanf("%d", &input);

        if (input == 0) {
            break; /* Dalja nga cikli */
        }

        printf("Numri juaj eshte: %d\n", input);
    }

    return 0;
}
```

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

Teknika e sentinelit

Sentineli është një vlerë e veçantë që sinjalizon fundin e input-it / ciklit.

- përdoret kur **s'dihet** sa elemente do të lexohen;
- e bën kushtin e ndalimit të thjeshtë (`n != SENTINEL`);
- kërkon që vlera sentinel të jetë **unikale** (të mos jetë e vlefshme si të dhënë).

Shembull: sentinel -1 për mbledhje serie

```c
#include <stdio.h>
#define SENTINEL -1

int main(void) {
    int n, sum = 0;

    printf("Fut numrat per t'i mbledhur (fut -1 per te perfunduar): ");
    scanf("%d", &n);

    while (n != SENTINEL) {
        sum += n;
        scanf("%d", &n);
    }

    printf("Shuma eshte: %d\n", sum);
    return 0;
}
```

Përmbajtja

1. Hyrje
2. Cikli `while`
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
5. `break`, `continue`, `goto`
6. Deklarata null

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

Sintaksa e do ... while

```
do deklarate;
while (shprehja);
```

Me trup bllok:

```
do {
    deklarate;
    deklarate;
} while (shprehja);
```

Detaj kritik

Ka **pikëpresje** pas `while (shprehja);`.

do vs while

Dallimi kryesor

- Te `do ... while`, trupi ekzekutohet **gjithmonë të paktën një herë**.
- Te `while`, trupi **mund të mos ekzekutohet fare**.

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

Shembull: numërim mbrapsht me do ... while

```c
#include <stdio.h>

int main(void) {
    int i = 10;

    do {
        printf("T minus %d and counting\n", i);
        --i;
    } while (i > 0);

    return 0;
}
```

Shembull: numri i shifrave (pse do është i duhuri)

```c
#include <stdio.h>

int main(void) {
    int digits = 0, n;

    printf("Fut nje numer jo-negativ: ");
    scanf("%d", &n);

    do {
        n /= 10;
        digits++;
    } while (n > 0);

    printf("Numri ka %d shifer(a).\n", digits);
    return 0;
}
```

Arsyeja

Edhe 0 ka të paktën **një shifër**; do garanton 1 ekzekutim minimal.

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

Validimi i input-it me while

```c
#include <stdio.h>

int main(void) {
    int n;

    printf("Fut nje numer jo-negativ: ");
    scanf("%d", &n);

    while (n < 0) {
        printf("Numri duhet te jete jo-negativ. Fut nje numer tjeter: ");
        scanf("%d", &n);
    }

    printf("Numri i futur eshte: %d\n", n);
    return 0;
}
```

Përmbajtja

1. Hyrje
2. Cikli `while`
3. Cikli `do ... while`
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`
   - C99: deklarim brenda `for`
   - Operatori i presjes

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

Sintaksa e for

Forma e përgjithshme:

```
for (shprehja1; shprehja2; shprehja3)
    deklarata;
```

Me trup bllok:

```
for (shprehja1; shprehja2; shprehja3) {
    deklarata;
    deklarata;
}
```

Kuptimi i 3 shprehjeve në `for`

Interpretimi standard

- **shprehja1 (Inicializimi)**: ekzekutohet **1 herë** para ciklit.
- **shprehja2 (Kushti)**: testohet **para çdo iteracioni**.
- **shprehja3 (Përditësimi)**: ekzekutohet **në fund** të çdo iteracioni.

Leximi praktik: *"Inicializo; për sa kohë kushti është TRUE, bëj trupin; pastaj përditëso dhe përsërit."*

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

Shembull: numërim mbrapsht me `for`

```c
#include <stdio.h>

int main(void) {
    int i;

    for (i = 10; i > 0; i--) {
        printf("T minus %d and counting\n", i);
    }

    return 0;
}
```

Shembull: numrat çift 2 deri 20

```c
#include <stdio.h>

int main(void) {
    int i;

    printf("Numrat cift nga 1 deri ne 20 jane:\n");
    for (i = 2; i <= 20; i += 2) {
        printf("%d\n", i);
    }

    return 0;
}
```

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

for është while i paketuar

Kjo formë:

```
for (shprehja1; shprehja2; shprehja3) {
    /* trupi */
}
```

është e barabartë me:

```
shprehja1;
while (shprehja2) {
    /* trupi */
    shprehja3;
}
```

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

Idiomat më të zakonshme të `for`

```c
/* 0 ... n-1 */
for (i = 0; i < n; i++) { /* ... */ }

/* 1 ... n */
for (i = 1; i <= n; i++) { /* ... */ }

/* n-1 ... 0 */
for (i = n - 1; i >= 0; i--) { /* ... */ }

/* n ... 1 */
for (i = n; i > 0; i--) { /* ... */ }
```

Gabime tipike në cikle

Gabimet më të shpeshta

1. Kusht i gabuar për drejtimin (numërim lart/poshtë).
2. Përdorimi i == si kusht cikli (shpesh s'ka kuptim).
3. Gabimet *off-by-one* (<= vs <).

Pyet veten: *"Cila është vlera e parë? Cila është e fundit? A përfshihet kufiri?"*

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

for fleksibël: shprehjet mund të mungojnë

Dy pikëpresjet ; ; **duhet të mbeten** gjithmonë.

```
/* pa inicializim (u be me pare) */
for (; i > 0; --i) { /* ... */ }

/* pa perditesim (u be brenda trupit) */
for (i = 10; i > 0; ) { i--; }

/* pa inicializim dhe pa perditesim */
for (; i > 0; ) { /* ... */ }

/* cikël i pafund */
for (;;) { /* ... */ }
```

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

C99: deklarimi i numëruesit brenda `for`

```c
for (int i = 0; i < n; i++) {
    /* kodi */
}
```

Avantazh

`i` ka **scope** vetëm brenda ciklit $\Rightarrow$ më pak gabime dhe kod më i pastër.

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

Operatori i presjes , (sidomos në `for`)

- `expr1, expr2` ekzekuton `expr1` dhe pastaj `expr2`;
- **vlera** e shprehjes është vlera e `expr2`;
- rendi është **majtas** → **djathtas**.

```
for (sum = 0, i = 1; i <= N; i++) {
    sum += i;
}
```

Lexueshmëria

Përdore kur e thjeshton realisht kodin (zakonisht në `for`), jo për "kompaktësi" artificiale.

Përmbajtja

1. Hyrje
2. Cikli `while`
3. Cikli `do ... while`
4. Cikli `for`
5. `break`, `continue`, `goto`
   - `break`
   - `continue`
   - `goto` (me kujdes)
6. Deklarata null

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

`break`: çfarë bën?

`break` transferon kontrollin **menjëherë** jashtë deklaratës më të afërt:

- `while`, `do ...  while`, `for`, `switch`

Detaj i rëndësishëm

`break` del vetëm nga **blloku më i afërt**. Në folezim, kjo ka shumë rëndësi.

Shembull: kontrolli i numrit të thjeshtë (dalje e parakohshme)

```c
#include <stdio.h>

int main(void) {
    int n, d;

    printf("Jep n: ");
    scanf("%d", &n);

    for (d = 2; d < n; d++) {
        if (n % d == 0)
            break; /* u gjet pjestuesi -> dil */
    }

    if (d < n)
        printf("%d eshte i pjestueshem me %d\n", n, d);
    else
        printf("%d eshte numer i thjeshte\n", n);

    return 0;
}
```

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

`continue`: çfarë bën?

`continue` **nuk del** nga cikli; shkurton iteracionin aktual:

- te `while`: kthehet te testimi i kushtit (në krye);
- te `do ... while`: shkon te testimi i kushtit (në fund);
- te `for`: shkon te përditësimi dhe pastaj te kushti.

Shembull: injoro vlerat zero me continue

```c
#include <stdio.h>

int main(void) {
    int n = 0; /* sa numra jo-zero kemi pranuar */
    int sum = 0;
    int x;

    while (n < 10) {
        scanf("%d", &x);
        if (x == 0)
            continue;

        sum += x;
        n++;
    }

    printf("Shuma e 10 numrave jo-zero eshte: %d\n", sum);
    return 0;
}
```

continue: kur ia vlen?

Kujdes

`continue` mund ta bëjë kodin më të shkurtër, por:

- me shumë kushte mund të ulë lexueshmërinë;
- përdore kur e bën logjikën më të qartë, jo thjesht më "kompakte".

Përmbajtja

1. Hyrje
2. Cikli `while`
   - Sintaksa dhe ideja
   - Shembuj bazë
   - Ciklet e pafundme dhe `break`
   - Sentineli
3. Cikli `do ... while`
   - Sintaksa dhe dallimi kryesor
   - Shembuj
   - Validimi i input-it
4. Cikli `for`
   - Sintaksa dhe kuptimi i tre shprehjeve
   - Shembuj tipikë
   - Marrëdhënia me `while`
   - Idiomat dhe gabimet tipike
   - Heqja e shprehjeve në `for`

goto: ideja bazë

```
label:
    deklarate;

goto label;
```

goto përdoret rrallë

Shmanget në kod modern sepse e bën rrjedhën të vështirë për t'u ndjekur. Megjithatë, mund të jetë praktik për:

- daljen nga cikle të folezuara;
- pastrim (cleanup) në trajtimin e gabimeve.

Shembull: goto për të dalë nga cikle të folezuara

```c
#include <stdio.h>

int main(void) {
    int i, j;

    for (i = 0; i < 5; i++) {
        for (j = 0; j < 5; j++) {
            if (i == 2 && j == 3)
                goto exit_loops;
            printf("i = %d, j = %d\n", i, j);
        }
    }

exit_loops:
    printf("Doli nga ciklet.\n");
    return 0;
}
```

Shembull: goto për cleanup në rast gabimi

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    FILE *fp = NULL;
    int *a = NULL;

    fp = fopen("data.txt", "r");
    if (fp == NULL) goto cleanup;

    a = malloc(100 * sizeof(int));
    if (a == NULL) goto cleanup;

    /* ... puna kryesore ... */

cleanup:
    if (a != NULL) free(a);
    if (fp != NULL) fclose(fp);

    return 0;
}
```

Përmbajtja

1. Hyrje
2. Cikli `while`
3. Cikli `do ... while`
4. Cikli `for`
5. `break`, `continue`, `goto`
6. Deklarata null
7. Mbyllje

Çfarë është deklarata null?

Në C, `;` është një deklaratë e vlefshme, por **pa efekt**.

Shembull i thjeshtë

```
i = 0; ; j = 1;
```

Këtu ka 3 deklarata: `i=0;`, një deklaratë null `;`, dhe `j=1;`.

Përdorimi kryesor: cikle me trup bosh

Kur e gjithë "puna" është në kontroll/përditësim:

```
for (d = 2; d < n && n % d != 0; d++)
    ; /* trup bosh */
```

Praktikë e mirë

Vendose ; në rresht më vete që të jetë e qartë se trupi është bosh.

Kurthi 1: `if me ;` të panevojshme

GABIM

```c
/* GABIM: printf ekzekutohet gjithmone */
if (d == 0);
    printf("Gabim: Pjestim me zero\n");
```

Arsyeja: trupi i `if` është deklarata null; `printf` nuk është pjesë e `if`-it.

Kurthi 2: `while` me trup bosh ⇒ cikël i pafund

GABIM

```
i = 10;
while (i > 0)
    ; /* trup bosh -> i nuk ndryshon -> cikli s'mbaron kurre */

{
    printf("T minus %d and counting\n", i);
    --i;
}
```

Kurthi 3: `for` me `;` në fund

GABIM

```c
for (i = 10; i > 0; i--)
    ; /* trup bosh */

printf("T minus %d and counting\n", i);
```

- cikli kryhet i tëri pa printuar asgjë;
- në fund `i == 0`;
- `printf` ekzekutohet vetëm **një herë**.

Përmbajtja

1. Hyrje
2. Cikli `while`
3. Cikli `do ... while`
4. Cikli `for`
5. `break`, `continue`, `goto`
6. Deklarata null
7. Mbyllje

Çfarë mësove në këtë kapitull

- konceptin e **ciklit** dhe rolin e kushtit/iteracionit;
- `while`: kushti testohet në fillim;
- `do ... while`: kushti testohet në fund (të paktën 1 herë);
- `for`: formë kompakte (inicializim/kusht/përditësim), idiomat dhe gabimet tipike;
- kontrollin e rrjedhës: `break`, `continue`, `goto` (me kujdes);
- **deklaratën null** `;` dhe kurthet e saj.

Si lidhet me kapitullin e mëparshëm dhe çfarë vjen më pas

- Kapitulli i mëparshëm: vendimmarrje (`if`, `switch`, ternar, direktiva kushtore).
- Ky kapitull: përsëritje + kontroll rrjedhe (cikle dhe dalje/kalime).
- Më pas (sugjerim tipik): **ciklet e folezuara**, **array**, dhe ushtrime me **algoritme** (p.sh. kërkim/numërim/filtrim).

Faleminderit për vëmendjen!

Pyetje?